A MINI PROJECT REPORT ON

SECURE VAULT & CREDENTIAL LEDGER

A Java Console-Based Credential Management Application

Submitted by

Rudrakshini Suresh, Surya Uddish
25311A67B2, 25311A6789
II - I

Table of Contents

1. Abstract and Introduction 2
2. Objectives and Technologies Used 2
3. System Design and Implementation 3
4. Testing, Limitations and Conclusion 5

1. Abstract and Introduction

1.1 Abstract

Secure Vault & Credential Ledger is a console-based Java application that lets a user keep a list of website credentials (website name, username and password) in one place. Access to the application is controlled by a Master PIN. Once inside, the user can add, view, search, edit and delete credentials, and can convert every stored password into scrambled text and back again using a simple Caesar-style character-shifting method. The project was built as a college mini project to demonstrate core Java concepts: classes and objects, encapsulation, the Array-List collection, methods, loops, conditional statements and basic encryption logic. All data is kept in memory only, so it is cleared when the program closes. The application is an educational demonstration and is not a production-ready password manager.

1.2 Introduction

Most people hold accounts on many websites, and each account has a username and a password. Remembering all of them is difficult, which often leads to weak or repeated passwords. A **credential manager** addresses this by storing account details in a single protected location and making them easy to find when required.

Two ideas are central to such a tool. The first is **authentication**: the user must prove they are allowed to open the vault, here by entering a PIN. The second is **encryption**: stored passwords should not be readable as plain text. This project shows both ideas at a beginner level. It uses a Master PIN for access and a Caesar-style shift to scramble passwords, so that a reader new to programming can follow exactly how each step works.

1.3 Purpose of the Project

The purpose of developing this application is to apply classroom Java knowledge to a small but complete program with a menu, user input validation and several cooperating classes. It also introduces the concept of protecting stored data, while being open about the fact that the technique used is only a teaching example (see Section 4.2).

2. Objectives and Technologies Used

2.1 Project Objectives

- To design a menu-driven console application that manages website credentials.
- To restrict access to the application using a Master PIN with a limited number of attempts.
- To implement add, view, search, edit and delete operations on stored credentials.
- To demonstrate password masking and a reversible Caesar-style encryption and decryption method.

- To practise object-oriented programming by separating data (`Credential`), encryption logic (`CipherEngine`) and program control (`VaultManager`).

2.2 Java Concepts Used

Concept	Where it is used	Purpose
Classes and objects	`Credential` objects are created for each account	Represent one saved account
Encapsulation	Private fields with getters and setters in `Credential`	Control access to account data
`ArrayList`	`vault` list in `VaultManager`	Store all credentials in memory
`Scanner`	Reads all keyboard input	Accept PIN, menu choices and details
Loops and `switch`	Menu loop and option handling	Repeat the menu until the user exits
Static methods	`CipherEngine.encrypt()` and `decrypt()`	Reuse encryption logic without an object
Exception handling	`NumberFormatException` in account-number input	Reject non-numeric entries safely

2.3 Development Environment

Item	Details
Programming language	Java (Java Development Kit installed on the system)
Editor	Visual Studio Code
Compile command	`javac *.java`
Run command	`java VaultManager`, or `java -jar SecureVault.jar` for the packaged JAR
Interface	Command-line (console) only
Java version used	Java JDK 27

3. System Design and Implementation

3.1 Architecture

The project is organised into three Java source files. `VaultManager` is the controlling class: it receives user input, shows the menu and calls the other two classes when required. `Credential` holds the details of a single account, and `CipherEngine` contains the logic that scrambles and restores text.

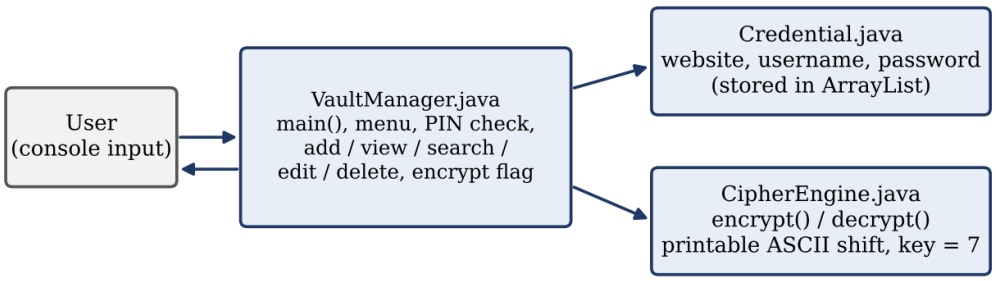


Figure 1: Structure of the application and relationship between the three classes

3.2 Application Workflow

When the program starts it asks for the Master PIN. After a correct PIN, the main menu is displayed repeatedly until the user chooses option 8. Each operation (options 1 to 7) is followed by a "Press Enter to continue" prompt, after which the menu is shown again. Any other input produces an "Invalid choice" message.

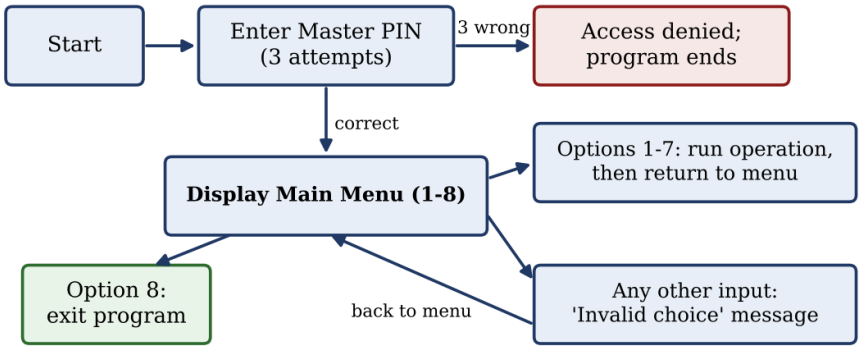


Figure 2: Workflow of the application from start-up to exit

3.3 Role of Each Java File

File	Role and main contents
`Credential.java`	Models one saved account. It has three private fields (`website`, `username`, `password`), a constructor, getters, setters, and the method `getMaskedPassword(boolean masked)`, which returns `********` when masking is requested and the real value otherwise.
`CipherEngine.java`	Provides two static methods, `encrypt(String, int)` and `decrypt(String, int)`, which shift each printable ASCII character forwards or backwards by the given key. Other characters are left unchanged.
`VaultManager.java`	Contains `main()`, the constants `MASTER_PIN` ("2206") and `ENCRYPTION_KEY` (7), the `ArrayList<Credential>` called `vault`, and a boolean flag `encrypted` that records whether the stored passwords are currently encrypted. It implements authentication, the menu and all credential operations.

3.4 Master PIN Authentication

The method `authenticate()` prompts for a PIN and compares it with the constant `MASTER_PIN`, which is set to `2206` in the source code. The user has **three attempts**; after each wrong entry the remaining attempts are displayed. If all three fail, the message "Access denied. Vault locked." is shown and the program ends without displaying the menu.

3.5 Credential Operations

The main menu offers eight options. The table below summarises how each one behaves in the implemented code.

No.	Option	Behaviour
1	View Vault Ledger	Displays all credentials in a formatted table with serial number, website, username and password column, followed by the total count. When the vault is decrypted the password column shows `********`; when encrypted it shows the scrambled text. A message is shown if the vault is empty.

No.	Option	Behaviour
2	Add New Credentials	Asks for website, username and password; empty input is rejected and re-requested. The new `Credential` is added to the `ArrayList`. Not allowed while the vault is encrypted.
3	Encrypt All Passwords	Encrypts every stored password using key 7 and sets the status to ENCRYPTED. Refused if the vault is empty or already encrypted.
4	Decrypt All Passwords	Restores every password to its original text and sets the status to DECRYPTED. Refused if the vault is not encrypted.
5	Search Credentials	Finds accounts whose website name equals the entered text (ignoring upper/lower case) and shows the website, username and masked or encrypted password.
6	Edit Credentials	Shows the ledger, asks for an account number, then replaces website, username and password with newly entered values. Not allowed while encrypted.
7	Delete Credentials	Shows the ledger, asks for an account number and removes that entry. Not allowed while encrypted.
8	Exit	Prints a goodbye message and ends the program.

Account numbers entered for editing or deletion are validated: non-numeric text or a number outside the range of saved accounts is rejected with a message, and no change is made. The status line "Ledger Status: ENCRYPTED / DECRYPTED" is shown beneath every menu.

3.6 Caesar-Style Encryption and Decryption

A Caesar cipher replaces each character by another character a fixed distance away. This project applies the idea to the 95 printable ASCII characters (codes 32 to 126, from the space character to the tilde). The distance is the key, which is fixed at **7**. Characters outside this range are copied unchanged.

Step	Formula used in the code
Encryption	`shifted = 32 + floorMod(character - 32 + shift, 95)`
Decryption	`shifted = 32 + floorMod(character - 32 - shift, 95)`
Key handling	`shift = floorMod(key, 95)`, which keeps the key within 0 to 94

The "wrap-around" behaviour of `floorMod` ensures that a character near the end of the range continues from the start of the range. For example, with key 7 the password `Hello123` becomes `Olssv89:`, because H→O, e→l, l→s, o→v, 1→8, 2→9 and 3→:. Likewise, `~` (code 126) wraps around to `&`. Decryption applies the same shift in the opposite direction and returns the original text exactly.

Encryption is performed by the user choosing option 3, not automatically. While the vault is encrypted, the add, edit and delete options are blocked so that scrambled and plain passwords are never mixed.

4. Testing, Limitations and Conclusion

4.1 Test Cases

The application was tested manually through the console by compiling the source files and running the menu options in sequence. The cases below were executed and each produced the expected result.

ID	Test case	Expected result	Outcome
T1	Enter wrong PIN three times	Remaining attempts shown each time; "Access denied"; program ends	Pass

ID	Test case	Expected result	Outcome
T2	Enter PIN `2206`	"Access granted" and main menu displayed	Pass
T3	Add credential (Gmail / user1 / Hello123)	Success message; account stored	Pass
T4	View ledger (decrypted)	Account listed with password shown as `********`	Pass
T5	Encrypt all passwords	Success message; status changes to ENCRYPTED	Pass
T6	View ledger (encrypted)	Password column shows `Olssv89:`	Pass
T7	Add credential while encrypted	Message asking user to decrypt first; nothing added	Pass
T8	Decrypt all passwords	Status changes to DECRYPTED; original password restored	Pass
T9	Search "gmail" (different case)	Matching account found with masked password	Pass
T10	Search a website not stored	"No matching account found."	Pass
T11	Edit account 1 with new details	"Credential updated successfully!"; ledger shows new values	Pass
T12	Enter invalid menu choice (e.g. 9)	"Invalid choice. Select 1-8."; menu shown again	Pass
T13	Delete account 1, then view ledger	Entry removed; "Vault is empty."	Pass
T14	Choose option 8	"Vault closed. Goodbye!" and program ends	Pass

4.2 Limitations

- **Hardcoded Master PIN:** the PIN `2206` is written in the source code and cannot be changed while the program is running.
- **Educational cipher only:** the Caesar-style shift with a fixed key of 7 is easily reversed and is not secure for real-world password storage.
- **In-memory storage:** credentials are held only in an `ArrayList` in memory; no file or database is used.
- **Data loss on exit:** all saved credentials are lost whenever the application closes.
- **Passwords are never displayed in readable form:** the view and search options always show masked or scrambled passwords, so the original password cannot be read back from within the application.
- **Basic search:** search matches the complete website name only, not partial text.
- **Console only:** the PIN is visible while it is typed, and there is no option to change the PIN.

4.3 Future Scope

The following are possible improvements only. **None of them is implemented in the current project.**

- Allowing the Master PIN to be set or changed by the user instead of being hardcoded.
- Saving credentials to a file so that data survives after the program closes.
- Using a stronger, standard encryption method in place of the Caesar-style shift.
- Adding partial-text search and an option to reveal a password after re-entering the PIN.

4.4 Conclusion

Secure Vault & Credential Ledger meets its objective as a beginner-level Java project. It combines PIN-based access, an in-memory list of credentials, password masking and a reversible character-shifting cipher in a clear, menu-driven program built from three cooperating classes. Developing it gave practical experience with encapsulation, collections, input validation and program flow. The project is intentionally simple: it stores data only in memory and uses a non-secure cipher, so it should be regarded as a learning demonstration rather than a real password manager. Even so, it provides a sound foundation on which the improvements listed in Section 4.3 could later be built.